

Module 3

Process Synchronization and Deadlocks

Concurrency in Operating System

Concurrency is the execution of the multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel. The running process threads always communicate with each other through shared memory or message passing. Concurrency results in sharing of resources result in problems like deadlocks and resources starvation.

It helps in techniques like coordinating execution of processes, memory allocation and execution scheduling for maximizing throughput.

Principles of Concurrency:

Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems.

The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Problems in Concurrency :

- **Sharing global resources –**
Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- **Optimal allocation of resources –**
It is difficult for the operating system to manage the allocation of resources optimally.
- **Locating programming errors –**
It is very difficult to locate a programming error because reports are usually not reproducible.
- **Locking the channel –**
It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency :

- **Running of multiple applications –**
It enable to run multiple applications at the same time.
- **Better resource utilization –**

It enables that the resources that are unused by one application can be used for other applications.

- **Better average response time –**

Without concurrency, each application has to be run to completion before the next one can be run.

- **Better performance –**

It enables the better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.

Drawbacks of Concurrency :

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency :

- **Non-atomic –**

Operations that are non-atomic but interruptible by multiple processes can cause problems.

- **Race conditions –**

A race condition occurs if the outcome depends on which of several processes gets to a point first.

- **Blocking –**

Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.

- **Starvation –**

It occurs when a process does not obtain service to progress.

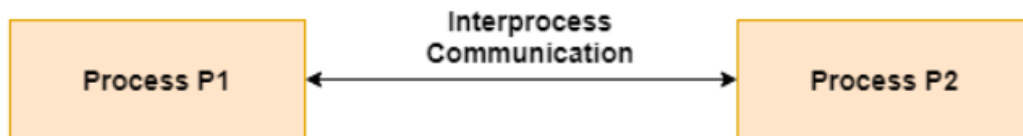
- **Deadlock –**

It occurs when two processes are blocked and hence neither can proceed to execute.

Inter Process Communication in OS

Inter Process Communication (IPC) refers to a mechanism, where the operating systems allow various processes to communicate with each other. This involves synchronizing their actions and managing shared data.

A diagram that illustrates interprocess communication is as follows –



There are 2 types of process –

- Independent Processes – Processes which do not share data with other processes .
- Cooperating Processes – Processes that shares data with other processes.

Cooperating process require Inter process communication (IPC) mechanism.

Inter Process Communication is the mechanism by which cooperating process share data and information.

There are 2 ways by which Inter process communication is achieved –

- Shared memory
- Message Passing

Shared Memory

1. A particular region of memory is shared between cooperating process.
2. Cooperating process can exchange information by reading and writing data to this shared region.
3. It's faster than Memory Parsing, as Kernel is required only once, that is, setting up a shared memory. After That, kernel assistance is not required.

Message Passing

1. Communication takes place by exchanging messages directly between cooperating process.

2. Easy to implement
3. Useful for small amount of data.
4. Implemented using System Calls, so takes more time than Shared Memory.

Process Synchronization

Process Synchronization means sharing system resources by processes in a such a way that, Concurrent access to shared data is handled thereby minimizing the chance of inconsistent data. Maintaining data consistency demands mechanisms to ensure synchronized execution of cooperating processes.

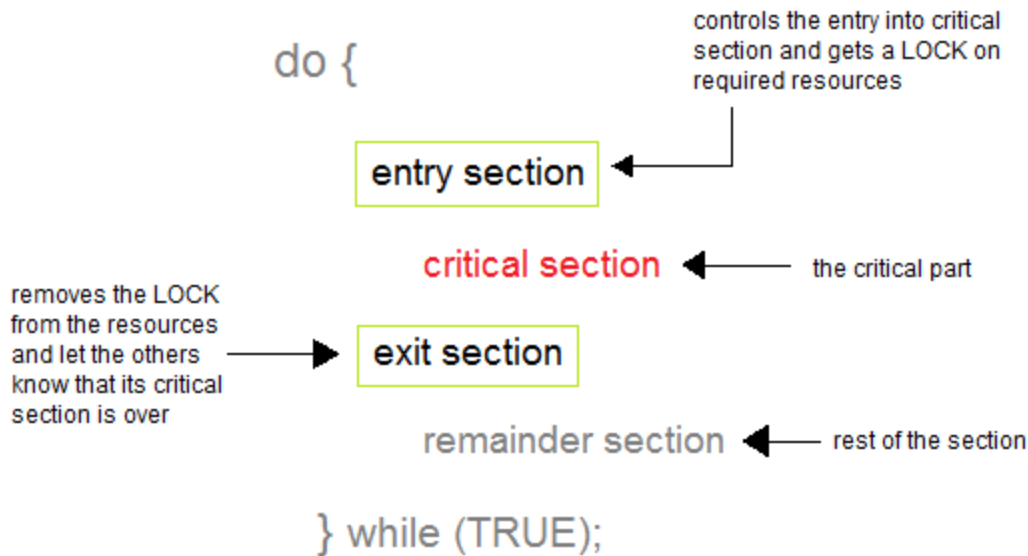
Process Synchronization was introduced to handle problems that arose while multiple process executions. Some of the problems are discussed below.

1. Critical Section Problem

2. Race Condition

1. Critical Section Problem

A Critical Section is a code segment that accesses shared variables and has to be executed as an atomic action. It means that in a group of cooperating processes, at a given point of time, only one process must be executing its critical section. If any other process also wants to execute its critical section, it must wait until the first one finishes.



This setup can be defined in various regions like:

- **Entry Section –**
It is part of the process which decide the entry of a particular process in the Critical Section, out of many other processes.
- **Critical Section –**
It is the part in which only one process is allowed to enter and modify the shared variable. This part of the process ensures that only no other process can access the resource of shared data.
- **Exit Section –**
This process allows the other process that is waiting in the Entry Section, to enter into the Critical Sections. It checks that a process that after a process has finished execution in Critical Section can be removed through this Exit Section.
- **Remainder Section –**
The other parts of the Code other than Entry Section, Critical Section and Exit Section are known as Remainder Section.

Solution to Critical Section Problem

A solution to the critical section problem must satisfy the following three conditions:

1. Mutual Exclusion

Out of a group of cooperating processes, only one process can be in its critical section at a given point of time.

2. Progress

If no process is in its critical section, and if one or more threads want to execute their critical section then any one of these threads must be allowed to get into its critical section.

3. Bounded Waiting

After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted. So after the limit is reached, system must grant the process permission to get into its critical section.

2. Race Condition

When more than one processes are executing the same code or accessing the same memory or any shared variable in that condition there is a possibility that the output or the value of the shared variable is wrong so for that all the processes doing the race to say that my output is correct this condition known as a race condition. Several processes access and process the manipulations over the same data concurrently, then the outcome depends on the particular order in which the access takes place.

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in the critical section differs according to the order in which the threads execute.

Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. That means that once a single thread is executing it, no other threads can execute it until the first thread has left the critical section. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.

Two Types of Race Conditions

Race conditions can occur when two or more threads read and write the same variable according to one of these two patterns:

- Read-modify-write
- Check-then-act

The **read-modify-write** pattern means, that two or more threads first read a given variable, then modify its value and write it back to the variable. For this to cause a problem, the new value must depend one way or another on the previous value. The problem that can occur is, if two threads read the value

(into CPU registers) then modify the value (in the CPU registers) and then write the values back.

The **check-then-act** pattern means, that two or more threads check a given condition, for instance if a Map contains a given value, and then go on to act based on that information, e.g. taking the value from the Map. The problem may occur if two threads check the Map for a given value at the same time - see that the value is present - and then both threads try to take (remove) that value. However, only one of the threads can actually take the value. The other thread will get a null value back. This could also happen if a Queue was used instead of a Map.

Preventing Race Conditions

To prevent race conditions from occurring you must make sure that the critical section is executed as an atomic instruction. That means that once a single thread is executing it, no other threads can execute it until the first thread has left the critical section.

Race conditions can be avoided by proper thread synchronization in critical sections. Thread synchronization can be achieved using a [synchronized block of Java code](#). Thread synchronization can also be achieved using other synchronization constructs like [locks](#) or atomic variables

Mutual Exclusion in Synchronization

During concurrent execution of processes, processes need to enter the [critical section](#) (or the section of the program shared across processes) at times for execution. It might so happen that because of the execution of multiple processes at once, the values stored in the critical section become inconsistent. In other words, the values depend on the sequence of execution of instructions – also known as a [race condition](#). The primary task of process synchronization is to get rid of race conditions while executing the critical section.

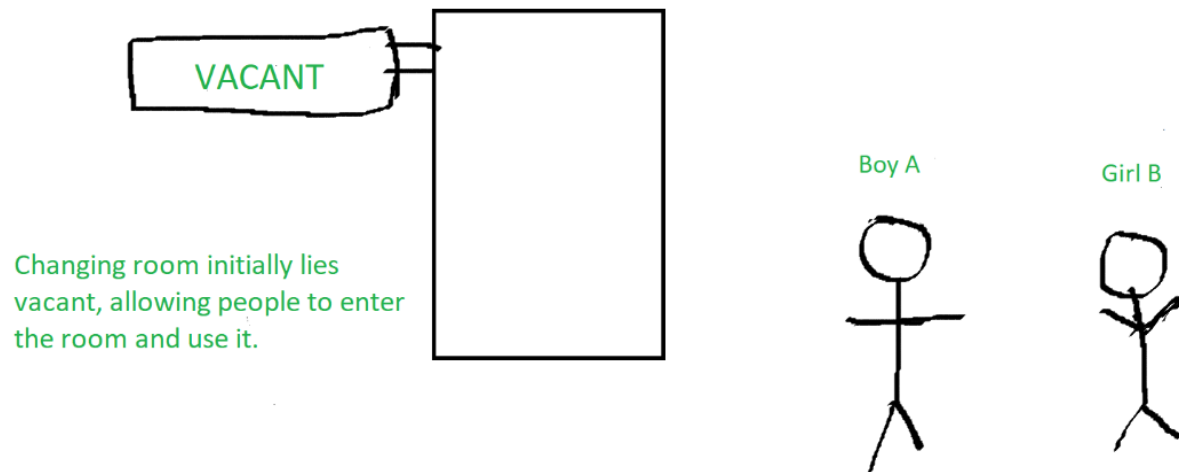
This is primarily achieved through [mutual exclusion](#).

Mutual exclusion is a property of [process synchronization](#) which states that “no two processes can exist in the critical section at any given point of time”. The term was first coined by Dijkstra. Any process synchronization technique being used must satisfy the property of mutual exclusion, without which it would not be possible to get rid of a race condition.

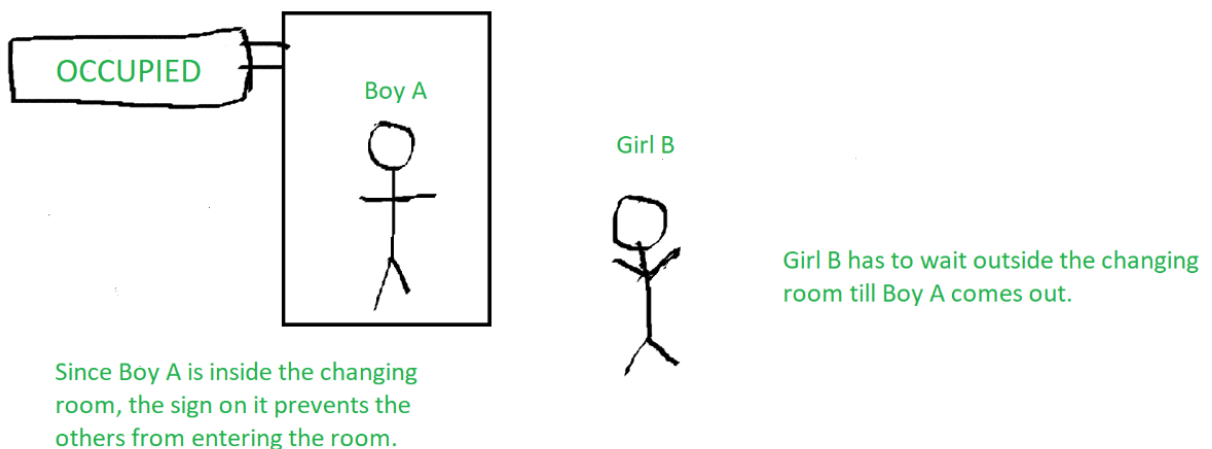
To understand mutual exclusion, let's take an example.

Example:

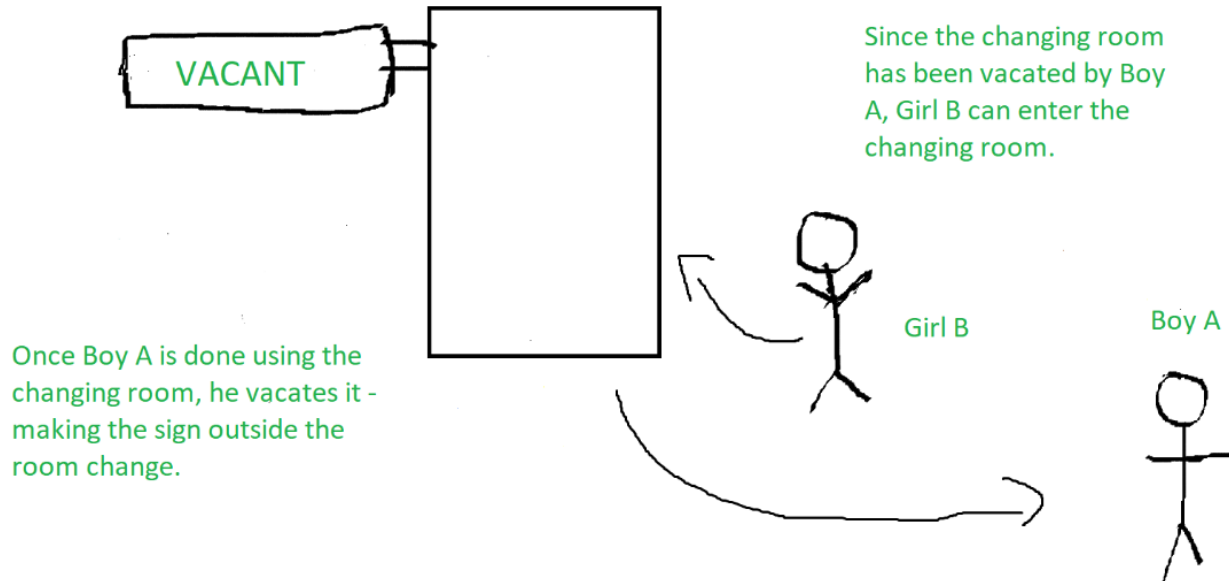
In the clothes section of a supermarket, two people are shopping for clothes.



Boy A decides upon some clothes to buy and heads to the changing room to try them out. Now, while boy A is inside the changing room, there is an 'occupied' sign on it – indicating that no one else can come in. Girl B has to use the changing room too, so she has to wait till boy A is done using the changing room.



Once boy A comes out of the changing room, the sign on it changes from 'occupied' to 'vacant' – indicating that another person can use it. Hence, girl B proceeds to use the changing room, while the sign displays 'occupied' again.



The changing room is nothing but the critical section, boy A and girl B are two different processes, while the sign outside the changing room indicates the process synchronization mechanism being used.

Synchronization Hardware

Many systems provide hardware support for critical section code. The critical section problem could be solved easily in a single-processor environment if we could disallow interrupts to occur while a shared variable or resource is being modified.

In this manner, we could be sure that the current sequence of instructions would be allowed to execute in order without pre-emption. Unfortunately, this solution is not feasible in a multiprocessor environment.

Disabling interrupt on a multiprocessor environment can be time consuming as the message is passed to all the processors.

This message transmission lag, delays entry of threads into critical section and the system efficiency decreases.

Mutex Locks

As the synchronization hardware solution is not easy to implement for everyone, a strict software approach called Mutex Locks was introduced. In this approach, in the entry section of code, a LOCK is acquired over the critical resources modified and used inside critical section, and in the exit section that LOCK is released.

As the resource is locked while a process executes its critical section hence no other process can access it.

Semaphores

Semaphore was proposed by Dijkstra in 1965 which is a very significant technique to manage concurrent processes by using a simple integer value, which is known as a semaphore. Semaphore is simply a variable which is non-negative and shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.

Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

The definitions of wait and signal are as follows –

- **Wait**

The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

```
wait(S)
{
    while (S<=0);

    S--;
}
```

- **Signal**

The signal operation increments the value of its argument S.

```
signal(S)
{
    S++;
}
```

Types of Semaphores

There are two main types of semaphores i.e. counting semaphores and binary semaphores. Details about these are given as follows –

- **Counting Semaphores**

These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources. If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.

- **Binary Semaphores**

The binary semaphores are like counting semaphores but their value is restricted to 0 and 1. The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes easier to implement binary semaphores than counting semaphores.

Advantages of Semaphores

Some of the advantages of semaphores are as follows –

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

Some of the disadvantages of semaphores are as follows –

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

Producer and Consumer problem

The Producer-Consumer problem is a classical multi-process synchronization problem, that is we are trying to achieve synchronization between more than one process.

There is one Producer in the producer-consumer problem, Producer is producing some items, whereas there is one Consumer that is consuming the items produced by the

Producer. The same memory buffer is shared by both producers and consumers which is of fixed-size.

The task of the Producer is to produce the item, put it into the memory buffer, and again start producing items. Whereas the task of the Consumer is to consume the item from the memory buffer.

Let's understand what is the problem?

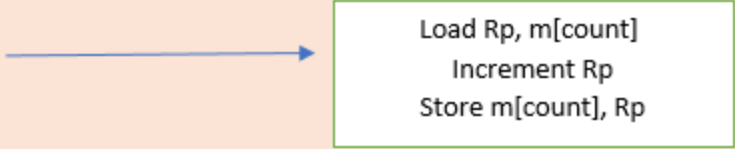
Below are a few points that considered as the problems occur in Producer-Consumer:

- The producer should produce data only when the buffer is not full. In case it is found that the buffer is full, the producer is not allowed to store any data into the memory buffer.
- Data can only be consumed by the consumer if and only if the memory buffer is not empty. In case it is found that the buffer is empty, the consumer is not allowed to use any data from the memory buffer.
- Accessing memory buffer should not be allowed to producer and consumer at the same time.

Let's see the code for the above problem:

Producer Code

```
int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}
```



Load Rp, m[count]
Increment Rp
Store m[count], Rp

Consumer Code

```

int count;
void consumer(void)
{
    int itemC;
    while( 1 )
    {
        while( count == 0 );
        itemC = buffer[ out ];
        out = ( out + 1 ) mod n;
        count = count - 1;
    }
}

```

Load Rc, m[count]
Decrement Rc
Store m[count]. Rc

Let's understand above Producer and Consumer code:

Before Starting an explanation of code, first, understand the few terms used in the above code:

1. "in" used in a producer code represent the next **empty buffer**
2. "out" used in consumer code represent first **filled buffer**
3. count keeps the count number of elements in the buffer
4. count is further divided into 3 lines code represented in the block in both the producer and consumer code.

If we talk about Producer code first:

--Rp is a register which keeps the value of m[count]

--Rp is incremented (As element has been added to buffer)

--an Incremented value of Rp is stored back to m[count]

Similarly, if we talk about Consumer code next:

--Rc is a register which keeps the value of m[count]

--Rc is decremented (As element has been removed out of buffer)

--the decremented value of Rc is stored back to m[count].

BUFFER

(pointer to the next empty space) $\rightarrow$ in = 5
(pointer to the first filled space) $\rightarrow$ out = 0
(number of elements in Buffer) $\rightarrow$ count = 5
(Size of Buffer) $\rightarrow$ n = 8

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

As we can see from Fig: Buffer has total 8 spaces out of which the first 5 are filled, in = 5 (pointing next empty position) and out = 0 (pointing first filled position).

Let's start with **the producer** who wanted to produce an element " F ", according to code it will enter into the producer() function, while(1) will always be true, itemP = F will be tried to insert into the buffer, before that while(count == n); will evaluate to be False.

Buffer[in] = itemP $\rightarrow$ Buffer[5] = F. (F is inserted now)

in = (in + 1) mod n $\rightarrow$ (5 + 1) mod 8 $\rightarrow$ 6, therefore in = 6; (next empty buffer)

After insertion of F, Buffer looks like this

Where **out = 0**, but **in = 6**

0	A
1	B
2	C
3	D
4	E
5	F
6	
7	

Since $\text{count} = \text{count} + 1$; is divided into three parts:

Load Rp, m[count] → will copy count value which is 5 to register Rp.

Increment Rp → will increment Rp to 6.

Suppose just after Increment and before the execution of third line (store m[count], Rp) **Context Switch** occurs and code jumps to **consumer code**. . .

Consumer Code:

Now starting **consumer** who wanted to consume the first element " A ", according to code it will enter into the consumer() function, while(1) will always be true, while(count == 0); will evaluate to be False(since the count is still 5, which is not equal to 0.

itemC = Buffer[out] → itemC = A (since out is 0)

out = (out + 1) mod n → (0 + 1) mod 8 → 1, therefore out = 1(first filled position)

A is removed now

After removal of A, Buffer look like this

Where **out = 1**, and **in = 6**

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Since $\text{count} = \text{count} - 1$; is divided into three parts:

Load Rc, m[count] → will copy count value which is 5 to register Rp.

Decrement Rc → will decrement Rc to 4.

store m[count], Rc → count = 4.

Now the current value of count is 4

Suppose after this **Context Switch** occurs back to the leftover part of producer code. .

Since context switch at producer code was occurred after Increment and before the execution of the third line (store m[count], Rp)

So we resume from here since Rp holds 6 as incremented value

Hence store m[count], Rp → count = 6

Now the current value of count is 6, which is wrong as Buffer has only 5 elements, this condition is known as Race Condition and Problem is Producer-Consumer Problem.

The solution of Producer-Consumer Problem using Semaphore

The above problems of Producer and Consumer which occurred due to context switch and producing inconsistent result can be solved with the help of semaphores.

To solve the problem occurred above of race condition, we are going to use **Binary Semaphore and Counting Semaphore**

Binary Semaphore: In Binary Semaphore, only two processes can compete to enter into its **CRITICAL SECTION** at any point in time, apart from this the condition of mutual exclusion is also preserved.

Counting Semaphore: In counting semaphore, more than two processes can compete to enter into its **CRITICAL SECTION** at any point of time apart from this the condition of mutual exclusion is also preserved.

Semaphore: A semaphore is an integer variable in S , that apart from initialization is accessed by only two standard atomic operations - wait and signal, whose definitions are as follows:

```
1. 1. wait( S )
2. {
3.  while( S <= 0 ) ;
4.  S--;
5. }
```

```
1. 2. signal( S )
2. {
3.  S++;
4. }
```

From the above definitions of wait, it is clear that if the value of $S \leq 0$ then it will enter into an infinite loop (because of the semicolon; after while loop). Whereas the job of the signal is to increment the value of S .

Let's see the code as a solution of producer and consumer problem using semaphore (Both Binary and Counting Semaphore):

Producer Code- solution

```
1. void producer( void )
2. {
3.  wait ( empty );
4.  wait(S);
5.  Produce_item(item P)
6.  buffer[ in ] = item P;
7.  in = (in + 1)mod n
8.  signal(S);
9.  signal(full);
```

10.

11.}

Consumer Code- solution

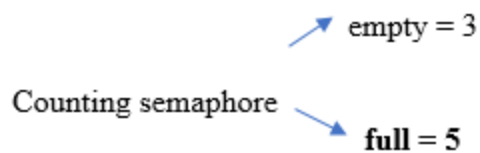
```
1. void consumer(void)
2. {
3.   wait ( empty );
4.   wait(S);
5.   itemC = buffer[ out ];
6.   out = ( out + 1 ) mod n;
7.   signal(S);
8.   signal(empty);
9. }
```

Let's understand the above Solution of Producer and Consumer code:

Before Starting an explanation of code, first, understand the few terms used in the above code:

1. "**in**" used in a producer code represent the next **empty buffer**
2. "**out**" used in consumer code represent first **filled buffer**
3. "**empty**" is counting semaphore which keeps a score of no. of empty buffer
4. "**full**" is counting semaphore which scores of no. of full buffer
5. "**S**" is a binary semaphore **BUFFER**

If we see the current situation of Buffer



S = 1(init. Value of Binary semaphore)

in = 5(next empty buffer)

out = 0(first filled buffer)

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

As we can see from Fig: Buffer has total 8 spaces out of which the first 5 are filled, in = 5(pointing next empty position) and out = 0(pointing first filled position).

Semaphores used in Producer Code:

6. wait(empty) will decrease the value of the counting semaphore variable **empty** by 1, that is when the producer produces some element then the value of the space gets automatically decreased by one in the buffer. In case the buffer is full, that is the value of the counting semaphore variable "empty" is 0, then wait(empty); will trap the process (as per definition of wait) and does not allow to go further.

7. wait(S) decreases the binary semaphore variable S to 0 so that no other process which is willing to enter into its critical section is allowed.

8. signal(s) increases the binary semaphore variable S to 1 so that other processes who are willing to enter into its critical section can now be allowed.

9. signal(full) increases the counting semaphore variable full by 1, as on adding the item into the buffer, one space is occupied in the buffer and the variable full must be updated.

Semaphores used in consumer Code:

10.wait(full) will decrease the value of the counting semaphore variable full by 1, that is when the consumer consumes some element then the value of the full space gets automatically decreased by one in the buffer. In case the buffer is empty, that is the value of the counting semaphore variable full is 0, then wait(full); will trap the process(as per definition of wait) and does not allow to go further.

11. wait(S) decreases the binary semaphore variable S to 0 so that no other process which is willing to enter into its critical section is allowed.

12. signal(S) increases the binary semaphore variable S to 1 so that other processes who are willing to enter into its critical section can now be allowed.

13. signal(empty) increases the counting semaphore variable empty by 1, as on removing an item from the buffer, one space is vacant in the buffer and the variable empty must be updated accordingly.

Producer Code:

Let's start with producer() who wanted to produce an element " F ", according to code it will enter into the producer() function.

wait(empty); will decrease the value of empty by one, i.e. $empty = 2$

Suppose just after this context switch occurs and jumps to consumer code.

Consumer Code:

Now starting consumer who wanted to consume first element " A ", according to code it will enter into consumer() function,

wait(full); will decrease the value of full by one, i.e. $full = 4$

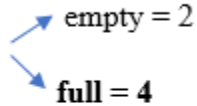
wait (S); will decrease the value of S to 0

itemC = Buffer[out]; $\rightarrow$ itemC = A (since out is 0)

A is removed now

$out = (out + 1) \bmod n \rightarrow (0 + 1) \bmod 8 \rightarrow 1$, therefore out = 1(first filled position)

current situation of Buffer

Counting semaphore 

0	
1	B
2	C
3	D
4	E
5	
6	
7	

$S = 0$ (Value of Binary semaphore)

in = 5(next empty buffer)

out = 1(first filled buffer)

Suppose just after this context, switch occurs back to producer code

Since the next instruction of producer() is wait(S);, this will trap the producer process, as the current value of S is 0, and wait(0); is an infinite loop: as per the definition of wait, hence producer cannot move further.

Therefore, we move back to the consumer process next instruction.

signal(S); will now increment the value of S to 1.

signal(empty); will increment empty by 1, i.e. empty = 3

current situation of Buffer

Counting semaphore $\begin{cases} \nearrow \text{empty} = 3 \\ \searrow \text{full} = 4 \end{cases}$
 S = 1 (Value of Binary semaphore)
 in = 5 (next empty buffer)
 out = 1 (first filled buffer)

0	
1	B
2	C
3	D
4	E
5	
6	
7	

Now moving back to producer() code;

Since the next instruction of producer() is wait(S); will successfully execute, as S is now 1 and it will decrease the value of S by 1, i.e. S = 0

Buffer[in] = itemP; $\rightarrow$ Buffer[5] = F. (F is inserted now)

in = (in + 1) mod n $\rightarrow$ (5 + 1) mod 8 $\rightarrow$ 6, therefore in = 6; (next empty buffer)

signal(S); will increment S by 1,

signal(full); will increment full by 1, i.e. full = 5

current situation of Buffer

Counting semaphore $\begin{cases} \nearrow \text{empty} = 3 \\ \searrow \text{full} = 5 \end{cases}$
 S = 1 (Value of Binary semaphore)
 in = 6 (next empty buffer)
 out = 1 (first filled buffer)

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Now add current value of full and empty, i.e. full + empty = 5 + 3 = 8 (which is absolutely fine) No inconsistent result is generated even after so many context

switches. But in the previous condition of producer and consumer without semaphore, we see the inconsistent result in case of context switches.

This is the solution to the Producer consumer problem.

THE DINING PHILOSOPHERS PROBLEM

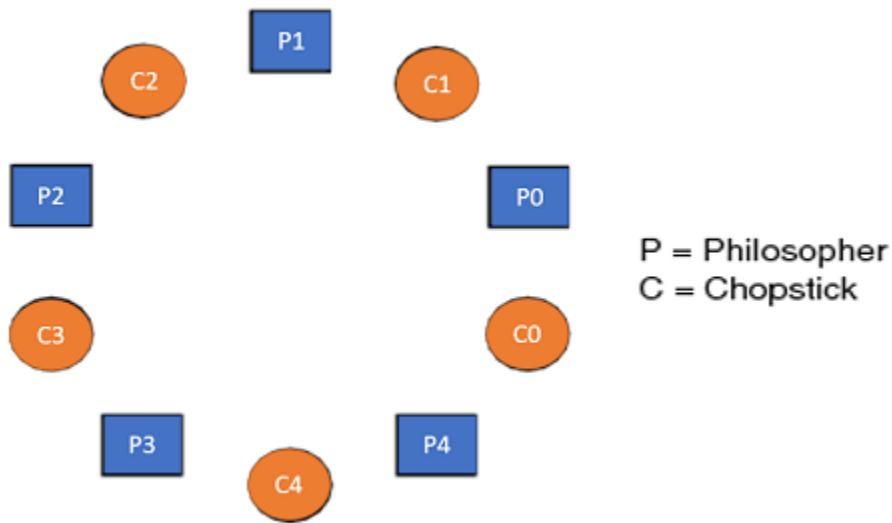
The dining philosopher's problem is the classical problem of synchronization which says that Five philosophers are sitting around a circular table and their job is to think and eat alternatively. A bowl of noodles is placed at the center of the table along with five chopsticks for each of the philosophers. To eat a philosopher needs both their right and a left chopstick. A philosopher can only eat if both immediate left and right chopsticks of the philosopher is available. In case if both immediate left and right chopsticks of the philosopher are not available then the philosopher puts down their (either left or right) chopstick and starts thinking again.

The dining philosopher demonstrates a large class of concurrency control problems hence it's a classic synchronization problem.



Five Philosophers sitting around the table

Dining Philosophers Problem- Let's understand the Dining Philosophers Problem with the below code, we have used fig 1 as a reference to make you understand the problem exactly. The five Philosophers are represented as P0, P1, P2, P3, and P4 and five chopsticks by C0, C1, C2, C3, and C4.



```

1. Void Philosopher
2. {
3.   while(1)
4.   {
5.     take_chopstick[i];
6.     take_chopstick[ (i+1) % 5] ;
7.     . .
8.     . EATING THE NOODLE
9.     .
10.    put_chopstick[i] );
11.    put_chopstick[ (i+1) % 5] ;
12.    .
13.    . THINKING
14.  }
15.}

```

Let's discuss the above code:

Suppose Philosopher P0 wants to eat, it will enter in Philosopher() function, and execute **take_chopstick[i];** by doing this it holds **C0 chopstick** after that it execute **take_chopstick[(i+1) % 5];** by doing this it holds **C1 chopstick** (since $i = 0$, therefore $(0 + 1) \% 5 = 1$)

Similarly suppose now Philosopher P1 wants to eat, it will enter in Philosopher() function, and execute **take_chopstick[i];** by doing this it holds **C1 chopstick** after

that it execute **take_chopstick[(i+1) % 5];** by doing this it holds **C2 chopstick**(since $i = 1$, therefore $(1 + 1) \% 5 = 2$)

But Practically Chopstick C1 is not available as it has already been taken by philosopher P0, hence the above code generates problems and produces race condition.

The solution of the Dining Philosophers Problem

We use a semaphore to represent a chopstick and this truly acts as a solution of the Dining Philosophers Problem. Wait and Signal operations will be used for the solution of the Dining Philosophers Problem, for picking a chopstick wait operation can be executed while for releasing a chopstick signal semaphore can be executed.

Semaphore: A semaphore is an integer variable in S, that apart from initialization is accessed by only two standard atomic operations - wait and signal, whose definitions are as follows:

1. 1. wait(S)
2. {
3. **while**(S <= 0) ;
4. S--;
5. }
- 6.
7. 2. signal(S)
8. {
9. S++;
10. }

From the above definitions of wait, it is clear that if the value of $S \leq 0$ then it will enter into an infinite loop(because of the semicolon; after while loop). Whereas the job of the signal is to increment the value of S.

The structure of the chopstick is an array of a semaphore which is represented as shown below -

1. semaphore C[5];

Initially, each element of the semaphore C0, C1, C2, C3, and C4 are initialized to 1 as the chopsticks are on the table and not picked up by any of the philosophers.

Let's modify the above code of the Dining Philosopher Problem by using semaphore operations wait and signal, the desired code looks like


```

1. void Philosopher
2. {
3.     while(1)
4.     {
5.         Wait( take_chopstickC[i] );
6.         Wait( take_chopstickC[(i+1) % 5] ) ;
7.         . .
8.         . EATING THE NOODLE
9.         .
10.        Signal( put_chopstickC[i] );
11.        Signal( put_chopstickC[ (i+1) % 5] ) ;
12.        .
13.        . THINKING
14.    }
15.}

```

In the above code, first wait operation is performed on `take_chopstickC[i]` and `take_chopstickC [ (i+1) % 5]`. This shows philosopher `i` have picked up the chopsticks from its left and right. The eating function is performed after that.

On completion of eating by philosopher `i` the, signal operation is performed on `take_chopstickC[i]` and `take_chopstickC [ (i+1) % 5]`. This shows that the philosopher `i` have eaten and put down both the left and right chopsticks. Finally, the philosopher starts thinking again.